

Context as a trained model: a language model that keeps no transcript and trains an expanding cascade of small models instead

Devieswar Kancheti¹

¹ [Department], [Institution], [City], [State or Province], [Country]

Corresponding Author:

Devieswar Kancheti¹

[Institution address, City, Postal code, Country]

Email address: [institutional email address]

Abstract

Background. A language model served through a stateless application programming interface (API) remembers nothing between calls, so a long session re-sends its whole transcript on every turn until the context window ends it. In deep-search sessions, where every turn returns a document, that limit arrives within a few hundred turns.

Methods. We replace the transcript with a context-mapping language model (CMLM): a small model that starts empty, is bound to the frontier model's rules and is trained as the person works. The frontier keeps no history. Before answering it asks the CMLM for the facts it needs through a recall tool, and afterwards it trains the CMLM through a teach tool. A CMLM that fills hands its oldest facts, as feature vectors rather than words, to a CMLM of its own, so the structure expands while every level stays small; an exact-key integer memory beside each level holds keyed facts verbatim. We prove that the request per turn is bounded for any session length, that the exact memory is revision-correct and that every capped level trains on all of its pairs, and we measure recall in a seeded synthetic world with a scripted frontier against a hidden truth.

Results. At 300 turns of 3,300-token documents, where the transcript path would send 983,286 tokens per question, the frontier read about 560 tokens per turn. A single CMLM's recall fell to 0.48 of its ceiling while the expanding tree held 0.82 and the cascade 0.67; over 20 seeds at 60 turns the tree and the cascade beat the single model by 0.099 (95% CI 0.067 to 0.131) and 0.071 (0.043 to 0.098). The exact memory answered all 12,440 keyed probes correctly. A model that learns the frontier's tool calls cut tokens per turn by 62% under a permissive gate but filed 54% of recovered facts under the wrong key, and under a predict-then-learn gate it never took over. Applied to a real 34-turn session of 243,167 tokens, the cost model gives 17,816 input tokens for the CMLM path against 831,269 token-equivalents for a cached transcript.

Conclusions. Context can be held as a trained, expanding model rather than as re-sent text, so the limit on what it holds becomes training rather than tokens. All recall results are synthetic and all token counts are estimates; the live path with a real frontier model remains to be measured.

Introduction

Every call to a frontier language model through an API starts blank. The caller sends the instructions, the tool definitions and the question, and for a conversation it also sends every earlier turn. Prompt caching makes the repeated prefix cheaper but not free, and the context window puts a hard ceiling on how much history a call can carry (Packer et al., 2023).

Compressed prefixes keep a per-call cost, and compression loses what it did not anticipate (Mu, Li & Goodman, 2023; Deng et al., 2024). In a deep-search session, where each turn brings back a document of thousands of tokens, the ceiling arrives within a few hundred turns and the session ends.

The usual answers keep the transcript and shrink it: a sliding window; summaries of earlier turns, bounded per level (Nijkamp et al., 2026); compression of context into latent vectors that a decoder is trained to read (Chevalier et al., 2023; Liu & Qiu, 2025; Li et al., 2026); retrieval over a store of past text (Khandelwal et al., 2020; Lewis et al., 2020); or memory tiers that the model manages through function calls (Packer et al., 2023; Yu et al., 2026). Each still treats context as text to be re-sent, and the latent variants need a decoder trained to read the latents, which a closed API does not offer. This paper asks a different question: what if the frontier model's context were not a transcript but a model?

We describe a context-mapping language model (CMLM). It is a small model that starts empty and is bound to the rules of the frontier model it serves, in the lineage of fast-weight memories (Schmidhuber, 1992), memory networks (Weston, Chopra & Bordes, 2015) and test-time memorisation (Hardt & Sun, 2023; Behrouz, Zhong & Mirrokni, 2024), but placed beside the frontier rather than inside it. The frontier keeps no transcript. Before answering it calls recall, asking the CMLM in its own words for what it needs; after answering it calls teach, handing the CMLM cue-and-fact pairs on which the CMLM takes gradient steps on local compute, at no token cost. What survives a turn lives in the CMLM's weights and store, and the CMLM is a file that can be carried between sessions and between frontier models. If the frontier is a model with a million-token window, its context is instead a small model that fills by training, so the limit on what it can hold is training rather than tokens.

A single CMLM forgets as it fills, the sequential-learning interference described by McCloskey & Cohen (1989), and replay (Rebuffi et al., 2017) only delays it. The relation between frontier and CMLM is therefore made recursive: a CMLM keeps a bounded number of facts and, for the rest, is itself the frontier of a CMLM of its own, handing down what it cannot hold. Between two CMLMs the cue travels as a feature vector, not as words; only the hop into the frontier is tokenized. A tree variant closes a full child and opens a new one under a routing parent, in the spirit of index-routed hierarchical memory (Sun & Zeng, 2025). Beside every level sits an exact-key integer memory, the Life component of living-fused (Kancheti, 2026), so keyed facts are held verbatim, revised with the newest value winning and answered with a structural abstention when unknown, which trained memories alone do not deliver on exact-recall tasks (V P, 2026).

The contributions of this paper are:

1. The CMLM loop: a frontier model with no transcript that pulls its context through a recall tool and trains that context through a teach tool.
2. Two expanding structures, a tree and a cascade, in which the CMLMs communicate in vectors, with recall measured against a falling ceiling as sessions grow from 30 to 300 turns.
3. The fusion of an exact-key integer memory with the trained map, measured for exactness, abstention and revision at every depth.
4. A tool-recovering model that learns the frontier's own tool calls in order to remove the tool definitions and the recall round trip, together with a predict-then-learn gate that keeps it from taking over before it is ready.
5. Formal guarantees for the mechanism (bounded input, exact revision, full replay under a cap, the interleave bound, the caching break-even and an exact, token-free vector hop) and a seeded synthetic world with a hidden truth, in which every reported number is computed by released code.

The experiments answer five research questions.

- **RQ1.** Can a small model trained only from a frontier's teach calls return, for a question in other words, the facts the question needs, and by how much does training beat the same model untrained?
- **RQ2.** Does bounding the store per model, by a tree or by a cascade, hold recall as the session grows where a single model's recall falls?
- **RQ3.** Does an exact-key integer memory fused at read time deliver exact recall, abstention and revision without changing the map's paraphrase recall or its token cost?
- **RQ4.** Can a model that learns the frontier's tool calls remove the tool definitions and the recall round trip, and under what gate is that safe?
- **RQ5.** At the scale the design is for, a session whose transcript passes a million tokens, what does the frontier read per turn and what recall do the expanding shapes keep?

The Guarantees subsection proves what can be proved about the mechanism; RQ1 and RQ2 concern the map's recall, which no theorem gives and the experiments settle. All recall results come from a synthetic world and a scripted frontier, token counts are estimates, and the live path with a real frontier model has not been run; the Limitations subsection lists these limits before the Conclusions draw on the numbers.

Related Work

Context as weights. Storing what a sequence has said in parameters rather than in an attention window goes back to fast-weight programmers (Schmidhuber, 1992) and memory networks (Weston, Chopra & Bordes, 2015), and has returned as test-time training and as architectures that update a memory module at inference (Hardt & Sun, 2023; Behrouz, Zhong & Mirrokni, 2024), including training-free associative memories (He et al., 2024). The CMLM is a deliberately small instance: two linear towers over hashed features (Weinberger et al., 2009), trained with a contrastive objective (Oord, Li & Vinyals, 2018) in the two-tower retrieval tradition (Huang et al., 2013). Its contribution is not the substrate but the loop around it, in which the frontier model itself writes the training pairs.

Retrieval. Retrieval-augmented generation (Lewis et al., 2020) and nearest-neighbour language models (Khandelwal et al., 2020) put a store beside a frozen model and read from it at inference. A CMLM is a retriever whose index is trained online on the session, and the Life beside it is an exact store. living-fused (Kancheti, 2026) supplies that store: a recency-dominant integer count table with structural abstention and a confidence-gated blend into a model's output distribution. Its integration notes name failure modes this work inherits and addresses: blending an organ that barely knows anything, writing questions into the store, and averaging values instead of overwriting them.

Latent links between models. Prefix-tuning (Li & Liang, 2021) and prompt tuning (Lester, Al-Rfou & Constant, 2021) show that a few dozen learned vectors can carry what many tokens carry when the receiving model was trained to read them; gist tokens (Mu, Li & Goodman, 2023) compress an instruction into a handful of such vectors; and the projectors of Flamingo (Alayrac et al., 2022) and LLaVA (Liu et al., 2023) feed a frozen language model inputs that are not text. Between two CMLMs this work already passes vectors. Into a closed frontier served by an API it cannot, because the API accepts tokens, so the last hop is words.

Forgetting and its measurement. Catastrophic forgetting in sequentially trained networks (McCloskey & Cohen, 1989) and replay as its standard remedy (Rebuffi et al., 2017) frame the CMLM's central problem. The finding reported here, that a model's recall of its own training cues stays near 1.0 while its recall of paraphrases falls, is a small instance of the gap between training fit and generalisation. The gate adopted for the tool-recovering model is prequential evaluation (Dawid, 1984): each example is predicted before it is learned.

API-side mechanisms. Prompt caching, server-side compaction that summarises earlier turns, context editing that clears old tool results, and deferred loading of tool definitions are the answers currently offered at the API boundary. All of them keep context as text. The cost analysis in the Discussion accounts for caching explicitly.

Closest work. No paper found in a search of arXiv on 25 September 2026 combines a frontier with no transcript, a separate small model trained through the frontier's own tool calls, a recursive cascade with vector links, and an exact integer memory fused at read time; each piece has

neighbours. MemGPT (Packer et al., 2023) has the model manage memory tiers through function calls, but the tiers hold text and the model keeps a working context. An architecture of levels, ticks and cascaded intelligence (Nijkamp et al., 2026) keeps continuity across context resets with bounded summaries per level; hierarchical memory with index-based routing (Sun & Zeng, 2025) resembles the tree's parent; and small language models have been given memory operations for agents (Zhang et al., 2026a). Context cascade compression (Liu & Qiu, 2025), AutoCompressors (Chevalier et al., 2023) and end-to-end context compression (Li et al., 2026) compress context into latent tokens for a decoder, the latent link that cannot be made into a closed API. Titans (Behrouz, Zhong & Mirrokni, 2024), test-time training on nearest neighbours (Hardt & Sun, 2023), test-time training by context distillation (Wang et al., 2026) and fast-weight product-key memory (Zhao & Jones, 2026) put memory into the model's own weights, whereas this work leaves the frontier untouched and trains a model beside it. OoO-Spec (Zhang et al., 2026b) predicts tool calls with a small sidecar model to make calling faster; the tool-recovering model here predicts them to remove the tool definitions, and reports the gate under which that is unsafe. A correction to surprisal-aware residual test-time training (V P, 2026) finds that such memories fail exact-recall tasks, which is the case for an exact organ beside a trained map.

Research gap. The literature divides into four roads, and each leaves the same question open. Memory inside the model's own weights (Hardt & Sun, 2023; Behrouz, Zhong & Mirrokni, 2024; Wang et al., 2026; Zhao & Jones, 2026) needs the frontier's parameters, which a served frontier does not expose. Memory as text tiers managed by the model (Packer et al., 2023; Sun & Zeng, 2025; Yu et al., 2026; Zhang et al., 2026a) still spends tokens re-reading context and cannot read past the window at answer time. Latent compression of context for a decoder trained to read it (Chevalier et al., 2023; Liu & Qiu, 2025; Li et al., 2026) requires that decoder, so it is closed to an API-served frontier. An exact store (Kancheti, 2026) recalls verbatim but cannot answer a paraphrase. No work found (i) trains a separate context model from the frontier's own tool calls with the frontier untouched, (ii) bounds forgetting by expanding that model into levels linked by vectors rather than by summarising, (iii) fuses an exact integer memory with a trained map at read time, (iv) reports recall against an explicit ceiling with an untrained and a token-window

baseline, or (v) measures when a model that has learned the frontier's tool calls is safe to take them over. Table 1 places the approaches side by side on the properties this gap turns on.

Table 1. Prior approaches to context beyond the window, on the properties this paper measures.

Approach	Where memory lives	Re-sent per turn	Past the window	Exact recall	Revision	Paraphrase	Frontier weights needed
Transcript (baseline)	the request	all history	no	yes, within the window	restate	yes	no
Sliding window	the request	last W tokens	loses the rest	within W only	restate	yes	no
Summarisation (Nijkamp et al., 2026)	summaries	summaries	yes	lossy	lossy	yes	no
Latent compression (Chevalier et al., 2023; Liu & Qiu, 2025; Li et al., 2026)	latent vectors	latents	yes	approximate	retrain	yes	yes, a trained decoder
Model-managed text tiers (Packer et al., 2023; Sun & Zeng, 2025; Yu et al., 2026)	text stores and working context	working context and retrieved text	yes	when retrieved	overwrite	via retrieval	no
Memory in weights (Hardt & Sun, 2023; Behrouz, Zhong & Mirrokni, 2024)	the model's parameters	nothing	yes	unreliable (V P, 2026)	none	yes	yes
Exact store (Kancheti, 2026)	integer table beside the model	readout	yes	yes	yes	no	no
This work	a trained model per level and an integer table	rules and readout, 430 to 560 tokens	yes	yes, the Life	yes, the Life	yes, the map	no

Re-sent per turn: what the request carries on every turn. Frontier weights needed: whether the approach needs access to, or training of, the serving model's parameters.

Materials & Methods

System overview

Figure 1 shows the whole system. Words pass only between the person and the frontier; the frontier reaches its context through two tools; levels of CMLMs hand facts down as vectors; a Life sits beside each level; and a tool-recovering model, once it is ready, makes the frontier's calls for it. The only tokenized hop inside the system is between the frontier and level 0, and it is tokenized only because a served API accepts nothing else.

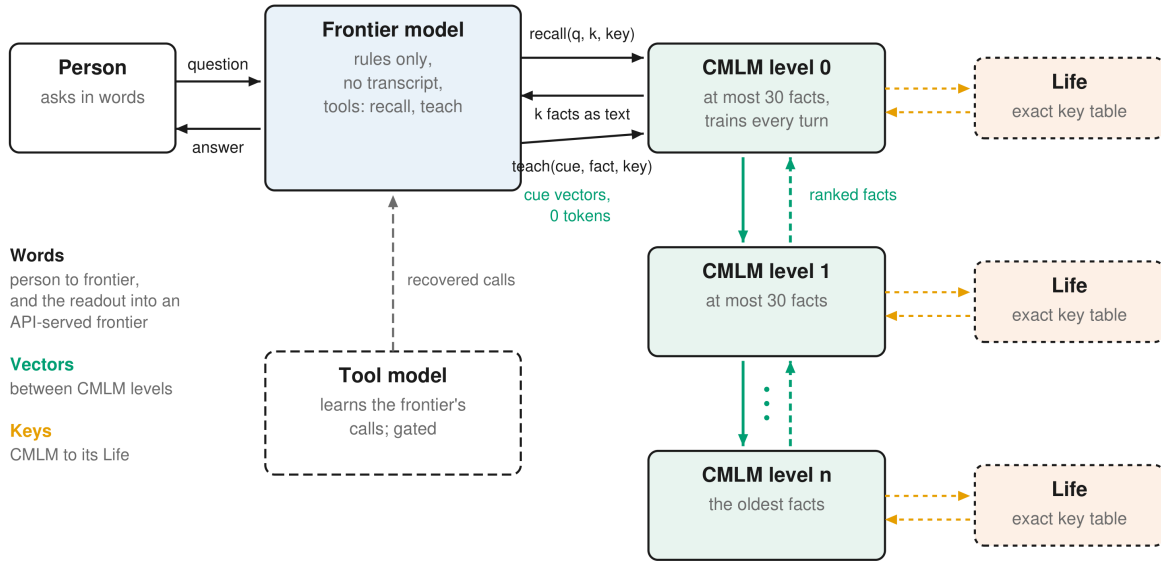


Figure 1. Architecture of the system. Words pass only between the person and the frontier model. The frontier keeps no transcript and reaches its context through two tools, recall and teach. CMLM levels hand their oldest facts down as feature vectors, at no token cost, and ranked facts climb back. An exact-key integer memory (the Life) sits beside every level. A tool-recovering model can make the frontier's calls once a prequential gate allows it.

The context-mapping language model

A CMLM maps a question to the facts it needs. Text is turned into a hashed feature vector of unigrams and bigrams with signs, normalised to unit length, in 4,096 slots, so that any word, seen or not, has a slot. Two towers project the features into a shared 64-dimensional space and score a query against a fact. At step zero both towers are the same random matrix, so the untrained model is a random projection of the words' cosine similarity; this untrained model is called matching and serves as a baseline, so that every comparison is the same model before and after training. Teaching a turn adds its cue-and-fact pairs and takes 20 steps of Adam on a batch of the new pairs plus up to 48 replayed old pairs, with a softmax over the facts in play at temperature 0.1, pulling each cue towards its fact and away from the others. Fact strings live in a store, because a hashed vector cannot be decoded; the weights hold what a store cannot, namely which facts a question never seen before should bring back. A readout is the top k facts by score. A 16-point grid of learning rate and temperature over eight seeds that are not used elsewhere moved final recall only between 0.63 and 0.72; the setting used, a learning rate of 0.05 and a temperature of 0.1, scored

0.671 against 0.721 for the best cell (a learning rate of 0.02 and a temperature of 0.5). The reported runs keep the original setting and were not re-tuned on this grid.

The loop

The frontier is given a system prompt of rules and two tools, and no history. Each turn's messages are the question, the tool calls with their results, and the answer, and they are dropped when the turn ends. The recall tool takes a query in the frontier's own words, a count k and an optional exact key; its result is the readout as text. The teach tool takes pairs of a cue in a person's words, a verbatim fact and an optional key. Both tools are declared with strict, closed schemas, so the frontier can only ever produce strings and a count. A message whose only tool call is teach ends the turn without a further call. If the frontier does not teach, a pattern teacher writes the pairs (the question as cue and the sentences that carry a number as facts) and the record says so. The transcript path is counted on every turn but never sent, so the two paths sit side by side in the record.

Expansion: the tree and the cascade

A single CMLM forgets as it fills, because training on new pairs moves the weights that held the old ones. Two structures bound the store per model.

The **tree** closes a child once it holds 30 facts; its weights never move again and a new child opens. A parent CMLM, whose facts are the child labels, learns from the same cues which child a question belongs to and ranks the children, and the readout interleaves their answers by rank. A self-measured trigger, closing a child when its recall of its own cues falls below 0.9, was also measured over 20 seeds at 60 turns: against the cap alone it changed recall at the last turn by -0.020 (95% CI -0.040 to +0.000) and was better on only two of 20 seeds, because a model fits its own cues while losing their paraphrases. The trigger is switched off in every other reported run.

The **cascade** makes the frontier-to-CMLM relation the same at every level. A CMLM keeps at most 30 facts and, for the rest, is the frontier of a CMLM of its own: after every teach it hands its oldest facts down, and the level below does the same. Between levels the cue travels as the feature vector already computed, and only the fact travels as a string, so nothing between

CMLMs is tokenized. Recall asks every level and interleaves the answers by rank, nearest level first. Every level replays all of its own pairs on every turn, so no level forgets within itself. Two alternatives were measured over the same 20 seeds: merging levels by a within-level normalised score changed recall at the last turn by -0.057 (95% CI -0.081 to -0.032) and was better on only one of 20 seeds, and handing a fact down as soon as its own cue stops returning it changed recall at the last turn by -0.047 (95% CI -0.075 to -0.020) and was better on only two of 20 seeds. Both remain as options in the code.

The exact-key memory beside the map

Beside any CMLM sits a Life: an exact-key, recency-dominant integer count table taken unchanged from living-fused (Kancheti, 2026). The teach tool's key writes the fact into it verbatim, and the recall tool's key reads it back verbatim or returns a structural ABSTAIN. When a key is asked for, the Life's counts are blended into the map's top k as a distribution with weight $t/(t + 0.25)$, the Life's own gate, so a single assertion by the frontier already speaks. A key that a model guessed, rather than one the frontier asked for, earns one slot (the Life's current value first) instead of the full blend. A revision is a second write under the same key: the newest value wins and the history is kept. Only teach writes the Life; recall never does. The store is integer and the floating-point blend happens only at read time, so the Life has a byte-exact identity that the map's weights cannot have.

The tool-recovering model

The tool definitions and the recall round trip exist only because the frontier decides what to ask and what to teach. A tool-recovering model with the CMLM's architecture learns those decisions from the frontier's own calls: one head maps a question to the key it asks for and another maps a stated sentence to the key it belongs under, each key by dotted part with one CMLM per position. While the frontier calls the tools, every keyed call is a training pair, predicted before it is learned so that a running prequential accuracy is kept. When that accuracy clears 0.85 on both heads after at least 30 keyed examples, the loop switches: the readout is fetched before the frontier is called, the frontier answers under short rules with no tools in one round trip, and the teach is recovered

from its answer. Two rules were forced by measurement: a label taught only once is a guess and is not predicted, and a sentence that restates the context sent or a fact already held is never taught back.

How the models communicate

Three links carry the whole system. **CMLM to frontier.** The frontier reaches its CMLM through recall and teach. On a frontier served through an API the boundary is tokens, because the API accepts nothing else, so the readout is rendered as k facts in text, each marked with the turn in which it was taught, and the teach pairs arrive as cue-and-fact strings with an optional key. That rendering is the only place in the system where tokens are counted. On an open-weights frontier the same link could be a trained projector into the model's embedding space; that variant is proposed, not built. **CMLM to CMLM.** A CMLM never sends words to another CMLM. When a level hands a fact down it sends the cue's 4,096-dimensional feature vector, which it has already computed, together with the fact string that the store must keep for the frontier, and the level below trains on the vector directly. A query descends the cascade as the same vector, computed once at the top, and every level scores it with its own towers; the readout climbs back as ranked facts. The weights are not shared: the vector is the common language because every CMLM hashes words the same way. **CMLM to Life.** Beside every CMLM the Life speaks exact strings: a key written by the frontier goes into the integer table with its fact, and at read time the Life's counts for that key are blended into the readout. The Life never sees the CMLM's vectors, and the CMLM sees the Life's counts only through that blend.

Formal statement

Table 2 lists the notation. Let a session be a sequence of turns $t = 1, 2, \dots$, each with a question q_t and an answer a_t from which the teacher extracts pairs (c, f) of a cue and a fact, and let F_t be the fact store after turn t .

Table 2. Notation.

Symbol	Meaning
q_t, a_t	the question and the answer of turn t
(c, f)	a cue and a fact: one teach pair
$F_t, F^{(\mathcal{L})}$	the fact store after turn t ; the store of level $\mathcal{L}$
$\varphi(x), D$	the hashed feature vector of a string; its width, 4,096
W_q, W_f, W_0, m	the query and fact towers, their shared initial matrix, and the shared width, 64
$s(q, f), \tau$	the score of a fact for a query; the softmax temperature, 0.1
B_t, N_t, R_t, C_t	the training batch, its new pairs, its replay sample and the candidate facts
k, n_i	facts asked for; facts that topic i holds
C	facts a level or child holds before it hands down or closes, 30
$\kappa, N, t_\kappa, w_\kappa, C_0$	a key, the Life's count table, the key's total count, its gate weight, and the gate constant 0.25
A_h, θ	a head's prequential accuracy; the switch threshold, 0.85
$R, Q_t, A_t, K, J, P_t, D_t$	tokens of the rules, the question, the answer, the readout, one tool call, the transcript so far and the last turn's delta
W, d, B	the context window; a lower bound on answer length; the bound on the CMLM path's input per turn

Features. Every string x is mapped to a fixed-width vector by hashing its unigrams and bigrams $G(x)$ into $D = 4096$ slots with a sign:

$$\varphi(x) = \frac{v(x)}{\|v(x)\|_2}, \quad v(x) = \sum_{g \in G(x)} \sigma(g) e_{h(g)}, \quad h: G \rightarrow \{1, \dots, D\}, \quad \sigma(g) \in \{-1, +1\}$$

Score. Two towers $W_q, W_f \in \mathbb{R}^{D \times m}$ with $m = 64$ project the features into a shared space, and at step zero $W_q = W_f = W_0$:

$$s(q, f) = \left(\varphi(q)^\top W_q \right) \cdot \left(\varphi(f)^\top W_f \right)$$

Training. Teaching turn t forms a batch B_t of its new pairs N_t and a replay sample R_t of at most 48 earlier pairs, and takes 20 Adam steps at learning rate 0.05 on a softmax over the candidate facts C_t (the facts of B_t plus up to 32 sampled from the store) at temperature $\tau = 0.1$:

$$\mathcal{L}_t = -\frac{1}{|B_t|} \sum_{(c,f) \in B_t} \log \frac{\exp(s(c,f)/\tau)}{\sum_{f' \in C_t} \exp(s(c,f')/\tau)}, \quad B_t = N_t \cup R_t$$

Readout and ceiling. A query returns the k highest-scoring facts. A probe on topic i needs n_i facts, so over all probes recall is bounded by a ceiling, and every late-session result is reported both raw and as a share of that ceiling:

$$\text{readout}_k(q) = \arg \top_k s(q, f), \quad \text{recall}@k \leq \frac{\sum_i \min(k, n_i)}{\sum_i n_i}$$

Cascade. Level ℓ keeps a store $F^{(\ell)}$ with $|F^{(\ell)}| \leq C = 30$. After each teach the oldest $|F^{(\ell)}| - C$ facts are handed to level $\ell + 1$ as pairs $(\varphi(c), f)$, and level $\ell + 1$ teaches on them and applies the same rule. A query descends as the single vector $\varphi(q)$, each level returns its own top k , and the readout is the rank interleave, nearest level first, truncated to k :

$$\text{out}_k(q) = \text{trunc}_k \left(\text{interleave}_{r=0,1,\dots} \left(\text{readout}_k^{(0)}(q)[r], \text{readout}_k^{(1)}(q)[r], \dots \right) \right)$$

Tree. A child closes at C facts and its towers are frozen. A parent CMLM with label facts ℓ_j is taught (c, ℓ_j) for every pair taught into child j , ranks the children by $s(q, \ell_j)$, and the readout interleaves the children's top k in that order.

Life. For key κ and value v the integer table N holds counts, and a write is recency-dominant, so the newest value wins while the history stays. An absent key returns ABSTAIN. At read time the counts are blended into the readout, p being the softmax of the readout's top- k scores, under the Life's own gate with $C_0 = 0.25$:

$$t_\kappa = \sum_v N[\kappa][v], \quad N[\kappa][v] \leftarrow N[\kappa][v] + t_\kappa + 1, \quad \text{recall}(\kappa) = \arg \max_v N[\kappa][v]$$

$$w_\kappa = \frac{t_\kappa}{t_\kappa + C_0}, \quad \tilde{p}(v) = (1 - w_\kappa)p(v) + w_\kappa \frac{N[\kappa][v]}{t_\kappa}$$

Tool-recovering model. Each head h predicts a key part before learning example i , giving a prequential accuracy over its n_h supervised examples, and the loop switches only when both heads clear $\theta = 0.85$ after at least 30 examples. A predicted part is emitted only if its softmax share is at least 0.6 and its label was taught at least twice.

$$A_h = \frac{1}{n_h} \sum_{i=1}^{n_h} \mathbb{1}[\hat{y}_i = y_i], \quad \text{switch} \iff \min(n_{\text{ask}}, n_{\text{state}}) \geq 30 \wedge A_{\text{ask}} \geq \theta \wedge A_{\text{state}} \geq \theta$$

Cost per turn. With R the rules, Q_t the question, A_t the answer, K the readout, J one tool call, $D_t = Q_{t-1} + A_{t-1}$ the delta that the last turn appended and $P_t = R + \sum_{i < t} (Q_i + A_i)$ the transcript, the transcript path sends $P_t + Q_t$ without a cache, or $0.1 (P_t - D_t) + 1.25 D_t + Q_t$ token-equivalents under a cache that reads at a tenth of the input price and writes at 1.25 times it, while the CMLM path with two uncached round trips sends

$$I_{\text{CMLM}}(t) = 2R + 2Q_t + K + J, \quad I_T(t) = P_t + Q_t$$

Algorithms. The three procedures that the paper measures are given as pseudocode.

```

Algorithm 1 One turn of the loop, with no transcript
Input: question q, CMLM M, Life L, frontier F with tools recall and teach
1. messages <- [q] # no history
2. loop:
3.   r <- F(rules, tools, messages)
4.   if r asks recall(query, k, key):
5.     rows <- M.readout(query, k)
6.     if key: rows <- L.answer(key, rows) # exact, ABSTAIN or blend
7.     messages <- messages + [r, result(rows)]; continue
8.   if r asks teach(pairs):
9.     for (c, f, key) in pairs:
10.      M.teach(c, f); if key: L.learn(key, f)
11.   break # teach-only ends the turn
12. return the answer text of r; drop messages

```

```

Algorithm 2 Cascade teach with hand-down
Input: level l with store F and cap C; level l+1, created when first needed
1. F.teach(pairs) # 20 Adam steps: new pairs + replay
2. while |F| > C:
3.   f <- oldest fact in F; V <- the cue vectors taught with f
4.   level(l+1).teach({(v, f) : v in V}) # vectors down, no words
5.   F.forget(f)
6. recall(query): v <- phi(query)
   return interleave_by_rank(level_0.top_k(v), level_1.top_k(v), ...)[:k]

```

```

Algorithm 3 Tool-recovering model with a prequential gate
Input: heads H_ask, H_state (empty CMLMs over key parts), warm-up 30, threshold theta

```

```

1. on a supervised turn with frontier calls:
2.   for each keyed recall (q, key):
      score H_ask.predict(q) == key; H_ask.learn(q, key)
3.   for each keyed teach (f, key):
      score H_state.predict(f) == key; H_state.learn(f, key)
4. ready <- n_ask >= 30 and n_state >= 30 and A_ask >= theta and A_state >= theta
5. if ready, on every later turn:
6.   key <- the literal key in q, else H_ask.predict(q)
      if its share >= 0.6 and its label was seen >= 2 times
7.   rows <- M.readout(q, k) with key           # a guessed key earns one slot
8.   answer <- F(short rules, no tools, [context(rows), q]) # one round trip
9.   for each sentence f in answer with a value that restates neither
      the context nor a held fact:
10.    M.teach(q, f); key' <- H_state.predict(f)
      if confident: L.learn(key', f)

```

Guarantees

This subsection proves what holds for the mechanism; what the map recalls is measured in the Results and is not a theorem. Let R be the tokens of the rules, $Q_t \leq Q_{\max}$ the question of turn t , A_t the answer, d a lower bound on the answers of a deep-search session (a document per turn), $K \leq k \ell_{\max}$ the readout for facts of at most $\ell_{\max}$ tokens, J a bound on the tokens of one tool call, and W the context window.

Theorem 1 (bounded input). Under Algorithm 1 the input sent in turn t satisfies, for every t ,

$$I_{\text{CMLM}}(t) \leq 2R + 2Q_{\max} + K + J =: B, \quad I_T(t) = R + Q_t + \sum_{i < t} (Q_i + A_i) \geq R + (t - 1)d,$$

and B does not depend on t .

Proof. Algorithm 1 builds every request from the question alone (line 1). The only additions before the answer are one recall call and its result (line 6), so the second round trip carries the rules, the question, the call and at most K tokens of readout, and no earlier turn is ever appended. Summing the two round trips gives B . The transcript path carries every earlier question and answer by definition, and each answer has at least d tokens. ■

Corollary 1 (the window). The transcript path exceeds any window W at the latest by turn

$$t^* = \left\lfloor \frac{W - R}{d + Q_{\min}} \right\rfloor + 1,$$

whereas the CMLM path fits every window $W \geq B$ for every t . With $W = 10^6$, $R = 164$, $d = 3300$ and $Q_{\min} = 10$, $t^* = 303$; the 300-turn run below reached 983,286 transcript tokens at $t = 300$ while the CMLM path sent about 560 tokens per turn throughout. This is the sense in which the design is proved for deep search: past the window the transcript path does not exist, and the CMLM path's request does not grow. ■

Theorem 2 (the Life is exact and revision-correct). With the write rule

$N[\kappa][v] \leftarrow N[\kappa][v] + t_\kappa + 1$, where t_κ is the total count under κ before the write, the value written last is the unique maximiser under κ ; an untaught key returns ABSTAIN; and no count ever decreases.

Proof. Let v^* be the value just written and t_κ the total before the write. Afterwards

$N[\kappa][v^*] \geq t_\kappa + 1 > t_\kappa \geq N[\kappa][v]$ for every other v , because t_κ is the sum of all counts under κ and each is non-negative. So $\text{recall}(\kappa) = v^*$ uniquely, whatever was written before and however often. A key that was never written has no entry, and recall returns ABSTAIN by construction. Counts are only ever incremented, so every earlier value remains in the table with its count. ■

Corollary 2. On any keyed probe of a taught key the Life's answer is the newest fact with probability 1, independent of the session's length, of the depth at which the fact's cue sits and of the state of the map. The measured 12,440 correct answers out of 12,440, and 10,330 out of 10,330 on revised keys, are this corollary observed.

Theorem 3 (full replay under the cap). If a level holds at most C pairs and the replay size ρ satisfies $\rho \geq C$, then every training step at that level optimises the objective over all of the level's pairs.

Proof. The batch is the new pairs together with a sample of $\min(\rho, n_{\text{old}})$ old pairs. With

$n_{\text{old}} \leq C \leq \rho$ the sample is the whole set, so B_t equals the level's entire pair set at every step. ■

Proposition 3 (the flat model's replay gap). A flat model holding $n > \rho$ pairs includes a given old pair in a teach with probability $\rho/(n - m)$, where m is the number of new pairs, so the expected number of consecutive teaches whose objective excludes that pair is $(n - m)/\rho - 1$, which

grows linearly in n . Each such teach moves the towers without that pair's constraint. This is the mechanism by which the flat model's recall of old facts falls with the session while the capped levels' recall does not. ■

Theorem 4 (the interleave guarantee). With L levels asked and a readout of size k , every needed fact ranked at a position $r < \lfloor k/L \rfloor$ within its own level appears in the readout, whatever its depth.

Proof. Rank interleaving emits rank 0 of every level, then rank 1 of every level, and so on, and stops at k facts. Ranks 0 to $\lfloor k/L \rfloor - 1$ of all L levels amount to $L \lfloor k/L \rfloor \leq k$ facts, so all of them are emitted before the cut. ■

Proposition 5 (the ceiling). For probes on topics holding n_i facts with k asked for, $\text{recall}@k \leq \sum_i \min(k, n_i) / \sum_i n_i$. *Proof.* A readout of k facts contains at most $\min(k, n_i)$ of the n_i facts needed. ■

Proposition 6 (break-even under caching). With cache reads at price ratio r and writes at w relative to uncached input, the warm transcript path costs $r(P_t - D_t) + w D_t + Q_t$ per turn and the CMLM path costs B uncached. The CMLM path is cheaper exactly when

$$P_t > \frac{B - Q_t - (w - r) D_t}{r}.$$

Proof. Compare the two costs and solve for P_t . With $r = 0.1$, $w = 1.25$, $Q_t \approx 15$ and $D_t \approx 50$, the threshold is about 3,600 tokens of transcript for the small world ($B \approx 430$) and about 4,900 for the document world ($B \approx 560$). Below it the transcript path is cheaper inside one warm session; above it, and in every cold session, the CMLM path is cheaper. The threshold falls as D_t grows: a turn that appends thousands of tokens must be written to the cache once at w , which alone exceeds B , so for sessions whose turns return documents the CMLM path is cheaper from the second turn on, with or without a cache. ■

Proposition 7 (the gate is honest). The prequential accuracy A_h is a mean of indicators $\mathbb{1}[\hat{y}_i = y_i]$ in which each $\hat{y}_i$ is a function of examples 1 to $i - 1$ only, so every term scores an

unseen example; recall of the taught cues scores the training set itself and can be 1 while A_h is far below it. *Proof.* By construction of Algorithm 3, lines 2 and 3: predict, then learn. ■

Proposition 8 (the vector hop is exact and free). Let a level hand a pair down as $(\varphi(c), f)$ rather than (c, f) . Then the level below reaches the same weights and the same readouts as it would from the words, and the hop carries no tokens and no decoding step.

Proof. Teaching on a string computes $\varphi(c)$ deterministically before the first gradient step, so the batch, the loss and every update are identical whether $\varphi(c)$ or c arrives, and the scores and readouts follow. A word hop would have to render the cue as text and encode it again, which between two models behind an API is a generated cue and a round trip; the vector hop is a copy of a vector that the level has already computed. Checked on 60 pairs of world seed 1: weights bit-identical, 63 of 63 readouts identical, maximum score difference 0.0, and 590 tokens that the cues would have cost as words against none. ■

Proposition 8 does not say that the vector carries more than the words: φ is a hash of the words, so it carries exactly what they carry and cannot be decoded back into them. The saving between CMLMs is the generation step and its tokens, not information. Any loss from putting a model's state into words arises only on the hop into the frontier, which this work still makes in words. Theorems 1 to 4 hold for any two-tower substrate, but whether the readout's top ranks contain the facts that a paraphrased question needs is a property of training and generalisation, which the experiments measure against a hidden truth.

Experimental design

The synthetic world. No third-party dataset is used. The world is generated from a seed and is byte-identical across runs (Table 3). It has ten topics, each with six fact words and six cue words, disjoint within a topic and across topics. A teaching turn asks about one topic in three of its cue words and is answered with one fact, stated in three of its fact words with a value (an amount, a schedule, a reference, a count or a deadline), between two filler sentences. In the paraphrase world a probe shares no word with the fact it needs; in the overlap world the cue words are the fact words. Each fact carries a key of the form topic.kind, so a key stated twice with a new value

is a natural revision. With bulk, every answer is padded with 250 filler sentences to a document of about 3,300 tokens that holds the same single fact, which is the deep-search shape. The only preprocessing is the hashing of words into features described above.

Table 3. The synthetic world and the settings used in every run unless a table says otherwise.

Setting	Value
Topics; cue and fact words per topic; value kinds	10; 6 and 6, all disjoint; 5
Facts per turn; filler sentences per answer; bulk filler	1; 2; 250 sentences, about 3,300 tokens
Possible keys (topic.kind)	50
Probes	every 5 turns, every topic taught so far, plus 3 never-taught keys with the Life
Feature width D; shared width m; temperature τ	4,096; 64; 0.1
Optimiser; learning rate; steps per teach; replay; extra negatives	Adam (0.9, 0.999); 0.05; 20; 48; 32
Readout k	4 at 30 turns; equal to facts per topic at 60 to 120 turns; 12 at 300 turns
Cap C per level or child; tree self-recall trigger	30; off
Life gate constant C0	0.25
Tool model: warm-up; gate θ ; part acceptance; minimum label count	30 keyed examples; 0.85 prequential; 0.6 softmax share; 2
Seeds	40 (30 and 60 turns, Table 4); 20 (60 to 120 turns, tool model, ablations); 5 (300 turns); 3 (key head alone); 8, seeds 41 to 48 (hyperparameter grid)
Hardware and software	Apple M4 Pro, 12 cores, 24 GB; macOS 26.5.2; Python 3.14.7; NumPy 2.4.4; no GPU

The scripted frontier. A scripted frontier stands in for the model: it calls recall with the question as the query, answers a teaching turn with the world's scripted text, quotes the readout back on a probe, and teaches the fact under its key. Probe scores therefore measure the CMLM's recall as the frontier would see it; the frontier's own judgement is not measured. This choice isolates the context model, which is the object of study, from the variability of a live model, and it is the main threat to external validity.

Probes, metrics and their justification. Every five turns, every topic taught so far is probed in other cue words, and the truth is that topic's facts so far; with the Life, every key taught so far is probed exactly and three keys that were never taught are probed beside them. Recall@k is the share of the facts a probe needs that appear in the frontier's answer. It is the right metric because

the frontier can use only what the readout returns, and it is read against the ceiling of Proposition 5, because with more facts per topic than k no readout can reach 1. Old-fact recall restricts recall to facts taught more than 15 turns before the probe and operationalises forgetting. For the Life, exact hit is the share of keyed probes on taught keys whose answer holds the newest fact, false abstain is the share of those that answered ABSTAIN, abstain is the share of probes on never-taught keys that answered ABSTAIN, and newest is exact hit restricted to keys stated more than once. Sent tokens count everything sent in a turn over all of its round trips.

Baselines. Matching is the same model at step zero. The window baseline is the last 320 tokens of transcript. The full transcript recalls everything at its full token cost by construction and is reported as a token count.

Token accounting. Token counts are characters divided by four, the estimate used throughout the code; the API's usage fields would be the truth and were not available. The transcript path is counted on every turn and never sent.

Runs and statistics. Seeds 1 to 40 are used for the 30- and 60-turn comparisons with the untrained baseline, seeds 1 to 20 for the 60- to 120-turn comparisons of shapes and for the tool model, seeds 1 to 5 for the 300-turn document runs, and seeds 1 to 3 for the key head tested alone. Means over seeds are reported with standard deviations and t-based 95% confidence intervals, and the shapes are compared with the flat model by per-seed paired differences on the same seed, which removes the variation between worlds.

Computing infrastructure. All runs used one Python process on an Apple M4 Pro (12 cores, 24 GB of memory) under macOS 26.5.2, with Python 3.14.7 and NumPy 2.4.4 and no GPU. Training and recall are real computations. Wall times are process wall-clock times measured while other runs shared the machine, so they are indicative only; every other number is deterministic for a given seed. The script that regenerates every table is supplied as Supplemental Code S1.

A human-driven frontier. With no API credential available, Claude Fable 5.1 (Anthropic) in an interactive session wrote the frontier's recall queries and keyed teach pairs for 12 turns of world

seed 1, then answered ten probes seeing only the readouts, scored against a truth that the answers never saw. The cue writing was not blind; the answering was. This is a sanity check, not a result.

A real transcript, costed. The conversation in which this work was developed is real data of the deep-search shape. Only its per-turn sizes are used (Supplemental Data S1): the number of characters in each of the person's 34 questions and the number appended by each turn. No message text is used or released.

Results

Training beats matching where the words differ (RQ1)

Over 40 seeds and 30 turns the trained CMLM recalled 0.662 of the facts that probes needed at the last turn, against 0.135 for the same model at step zero and 0.228 for a 320-token window, and it beat the untrained model on 40 of 40 seeds (Table 4). Where the probe's words overlap the fact's, matching already reaches 0.403 and training adds 0.209 rather than 0.527. The frontier was sent 429 tokens per turn over two round trips while the transcript path reached 1,584, and training took 27 ms per turn.

Table 4. Short sessions: the trained CMLM against the same model at step zero and a token window, 40 seeds.

Measure	30 turns, k = 4, paraphrase	30 turns, k = 4, overlap	60 turns, k = 4, paraphrase
Trained CMLM, recall at the last turn (fraction)	0.662	0.612	0.416
Matching, step zero (fraction)	0.135	0.403	0.060
Last 320 tokens of transcript (fraction)	0.228	0.225	0.114
Ceiling at the last turn (fraction)	1.000	1.000	0.667
Seeds on which training beat matching (count of 40)	40	38	40
Old facts, trained (fraction)	0.698	0.639	0.525
Recent facts, trained (fraction)	0.835	0.799	0.817
Transcript path at the last turn (tokens)	1,584	1,591	2,996
Sent to the frontier per turn (tokens)	429	430	429
Training time (ms per turn)	27	26	30

Recall is pooled over the probes of the last turn. Old facts were taught more than 15 turns before the probe; the old and recent values pool all probes of the session. Token counts are characters divided by four.

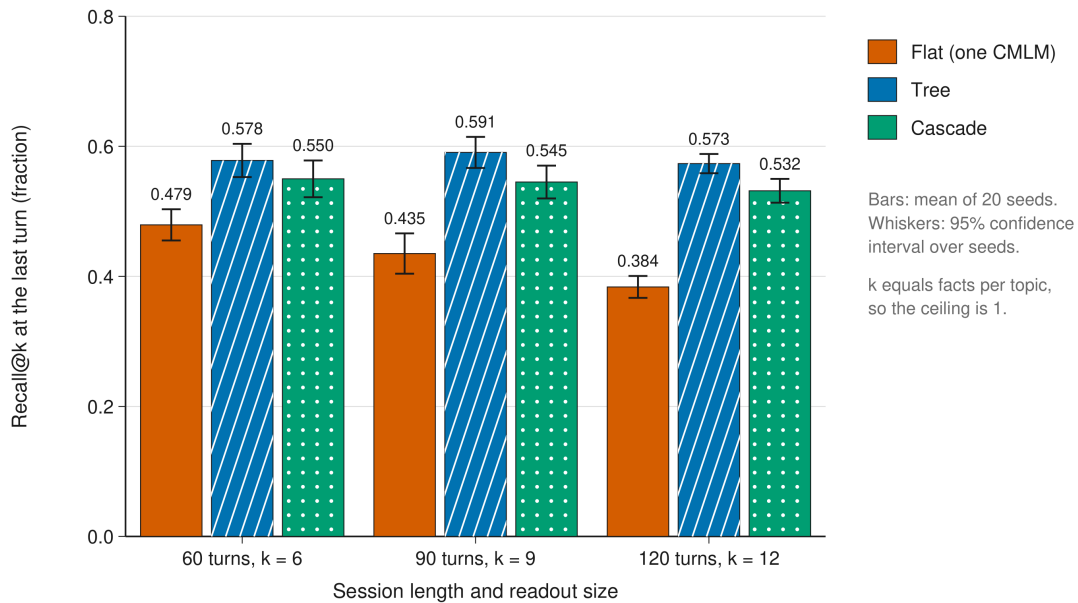
Expansion holds recall where the flat model forgets (RQ2)

As sessions doubled from 60 to 120 turns, the flat model's recall at the last turn fell from 0.479 to 0.384, while the cascade stayed between 0.532 and 0.550 and the tree between 0.573 and 0.591 (Table 5, Fig. 2). Old facts show the mechanism: at 120 turns the flat model recalled 0.514 of facts taught more than 15 turns before the probe against 0.950 of recent ones, while the tree held old facts at 0.651 and the cascade at 0.616. The tree's closed children were trained only on what they hold and then frozen; the cascade's top level keeps training, so its weights still carry facts it has handed down. Sent tokens per turn at 120 turns were 557 to 559 against a transcript path of 5,834.

Table 5. Recall as sessions grow: the flat model against the tree and the cascade, 20 seeds.

Session	Shape	Recall at the last turn, mean (SD) (fraction)	95% CI (fraction)	Old facts (fraction)	Recent facts (fraction)	Sent per turn (tokens)	Training (ms per turn)
60 turns, k = 6	Flat	0.479 (0.051)	0.455 to 0.503	0.606	0.872	461	44
60 turns, k = 6	Tree	0.578 (0.055)	0.553 to 0.604	0.638	0.865	460	78
60 turns, k = 6	Cascade	0.550 (0.060)	0.522 to 0.578	0.620	0.862	461	58
90 turns, k = 9	Flat	0.435 (0.066)	0.404 to 0.466	0.558	0.928	509	46
90 turns, k = 9	Tree	0.591 (0.051)	0.567 to 0.615	0.653	0.932	510	73
90 turns, k = 9	Cascade	0.545 (0.054)	0.520 to 0.570	0.623	0.926	512	73
120 turns, k = 12	Flat	0.384 (0.036)	0.367 to 0.401	0.514	0.950	557	48
120 turns, k = 12	Tree	0.573 (0.032)	0.559 to 0.588	0.651	0.956	557	72
120 turns, k = 12	Cascade	0.532 (0.039)	0.513 to 0.550	0.616	0.947	559	78

k equals the facts per topic at the last turn, so the ceiling is 1. Old facts were taught more than 15 turns before the probe; old and recent values pool all probes of the session.

**Figure 2.** Recall at the last turn as sessions grow, by shape. Bars are means over 20 seeds and whiskers are 95% confidence intervals over seeds. k equals the facts per topic at the last turn, so the ceiling is 1. Hatching distinguishes the shapes without colour.

A million tokens deep (RQ5)

At 300 turns of 3,300-token documents the transcript path would send 983,286 tokens per question, beyond a million-token window by Corollary 1; the frontier was sent about 560 tokens per turn throughout (Table 6, Fig. 4). Read as a share of a ceiling that falls through the session,

the flat model slid from 0.68 at turn 50 to 0.48 at turn 300, and by the end its recall of the cues it had itself been taught had fallen to 0.814, against 0.993 for the tree and 0.994 for the cascade. Over the same checkpoints the tree stayed between 0.60 and 0.82 of its ceiling and the cascade between 0.57 and 0.76 (Fig. 3), because no level holds more than 30 facts and every level replays all of them. A 320-token window holds none of these documents and recalled nothing. The Life was exact on all 12,440 keyed probes at every depth.

Table 6. A million tokens deep: 300 turns of 3,300-token documents, five seeds, each shape with the Life.

Measure	Flat + Life	Tree + Life	Cascade + Life
Recall@12 at the last turn, mean (SD) (fraction)	0.237 (0.035)	0.324 (0.014)	0.301 (0.020)
Share of the ceiling at turns 50, 150 and 300, seed 1 (fraction)	0.68, 0.47, 0.48	0.78, 0.65, 0.82	0.76, 0.60, 0.67
Old facts, mean (SD) (fraction)	0.320 (0.004)	0.436 (0.004)	0.408 (0.010)
Recall of its own taught cues at the last turn (fraction)	0.814	0.993	0.994
Life: exact on keyed probes (fraction of probes)	1.000 of 12,440	1.000 of 12,440	1.000 of 12,440
Life: ABSTAIN on unknown keys (fraction of probes)	1.000 of 900	1.000 of 900	1.000 of 900
Life: newest value on revised keys (fraction of probes)	1.000 of 10,330	1.000 of 10,330	1.000 of 10,330
Sent per turn (tokens)	560	558	560
Transcript path at turn 300 (tokens)	983,286	983,286	983,286
Models at the end (count)	1 model	11 children	10 levels
Training per session (s)	26.4	35.7	95.2

The ceiling of recall@12 at the last turn is 0.400. Token counts are characters divided by four; the transcript path is counted, never sent.

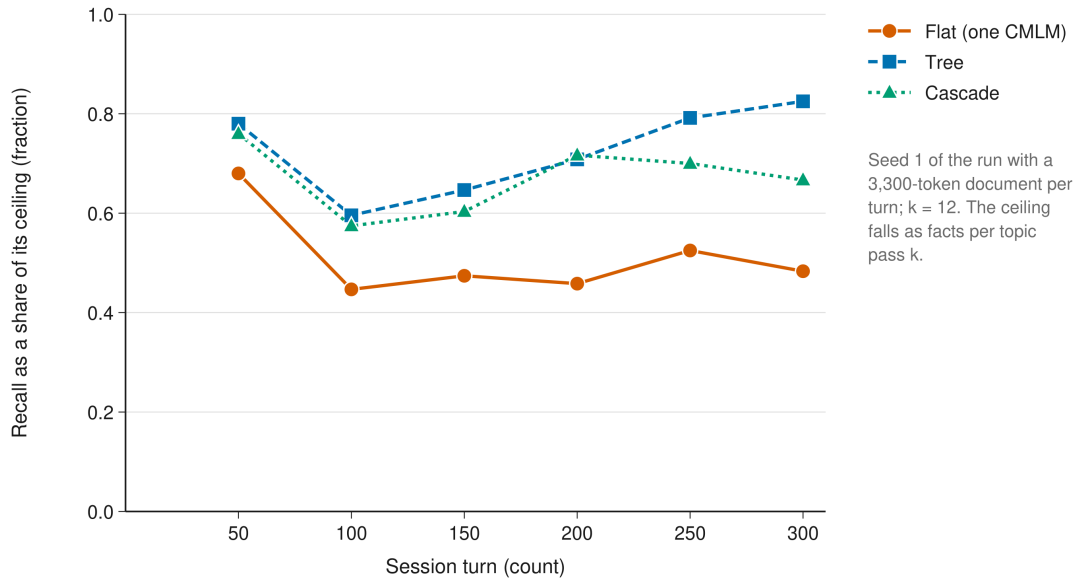


Figure 3. Recall as a share of its ceiling through a 300-turn session. Seed 1 of the run with a 3,300-token document per turn and $k = 12$; markers are the probe checkpoints every 50 turns. The ceiling falls as facts per topic pass k .

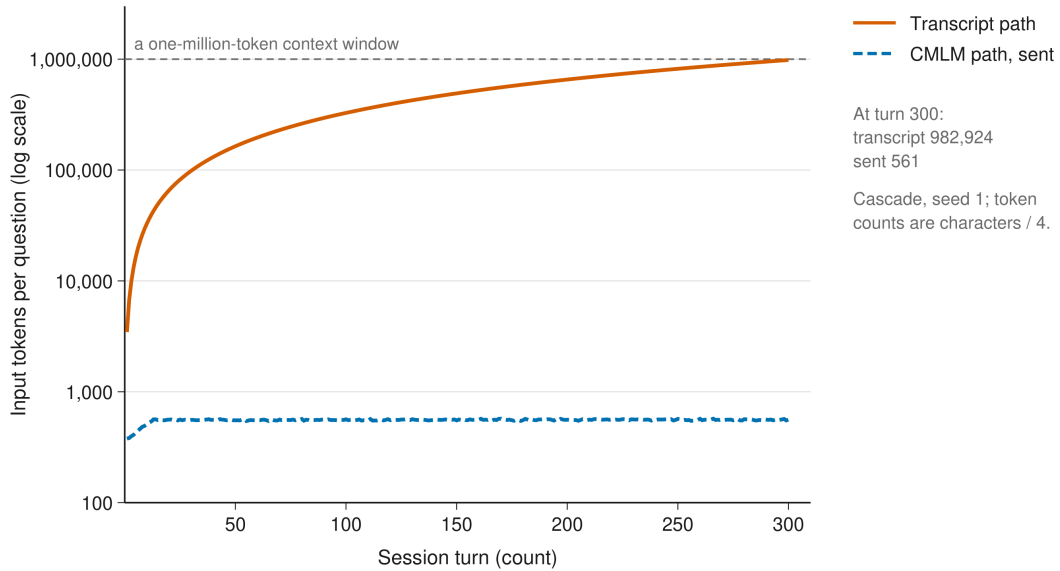


Figure 4. Input tokens per question through the 300-turn session. Log scale. The transcript path is counted on every turn and never sent; the CMLM path is what the frontier was actually sent. Cascade, seed 1; token counts are characters divided by four. The dashed line marks a one-million-token context window.

Statistical reliability

The central comparison, expansion against a single model, holds with the seeds separated (Table 7). Over 20 seeds at 60 turns with $k = 6$, the tree's recall at the last turn exceeded the flat model's by 0.099 (95% CI 0.067 to 0.131), winning on 18 of 20 seeds, and the cascade's by 0.071 (0.043 to 0.098), winning on 17 of 20. Tokens sent per turn were 460 to 461 for the three shapes, so the gain is not bought with a larger readout. At 300 turns, with five seeds, the tree beat the flat model by 0.087 (0.049 to 0.126), winning on five of five seeds, and the cascade by 0.065 (0.024 to 0.105), winning on five of five (Table 8). Old-fact recall separates the shapes most sharply. Five seeds give wide intervals, and the paired rows are the stronger evidence.

Table 7. Recall at the last turn and old-fact recall at 60 turns, $k = 6$, 20 seeds: mean, standard deviation, 95% confidence interval, and the paired difference against the flat model.

Shape	Recall, mean (SD) (fraction)	95% CI (fraction)	Old facts, mean (SD) (fraction)	95% CI (fraction)	Gain over flat, mean [95% CI] (fraction)	Seeds won (count)	Sent per turn (tokens)
Flat	0.479 (0.051)	0.455 to 0.503	0.606 (0.042)	0.586 to 0.625			461
Tree	0.578 (0.055)	0.553 to 0.604	0.638 (0.031)	0.624 to 0.653	+0.099 [0.067, 0.131]	18 of 20	460
Cascade	0.550 (0.060)	0.522 to 0.578	0.620 (0.034)	0.604 to 0.635	+0.071 [0.043, 0.098]	17 of 20	461

Intervals are t-based on per-seed values; the gains are per-seed differences against the flat model on the same seed.

Table 8. The same statistics for the 300-turn document run, $k = 12$, five seeds.

Shape	Recall, mean (SD) (fraction)	95% CI (fraction)	Old facts, mean (SD) (fraction)	95% CI (fraction)	Gain over flat, mean [95% CI] (fraction)	Seeds won (count)	Sent per turn (tokens)
Flat	0.237 (0.035)	0.193 to 0.281	0.320 (0.004)	0.316 to 0.325			560
Tree	0.324 (0.014)	0.307 to 0.341	0.436 (0.004)	0.430 to 0.441	+0.087 [0.049, 0.126]	5 of 5	558
Cascade	0.301 (0.020)	0.276 to 0.327	0.408 (0.010)	0.396 to 0.420	+0.065 [0.024, 0.105]	5 of 5	560

The ceiling of recall@12 at the last turn is 0.400. Intervals are t-based on per-seed values; the gains are per-seed differences against the flat model on the same seed.

The exact memory changes nothing about the map (RQ3)

With the Life attached, over 20 seeds and 60 turns with $k = 6$, every keyed probe was answered exactly, every unknown key drew ABSTAIN, and every revised key returned its newest value, in the flat, tree and cascade forms alike (Table 9). Paraphrase recall was identical to the runs without the Life, and tokens sent differed by at most 0.7 per turn, the key field that a tool call gains. In the

tree the earlier value of a revised key often sits in a closed child whose weights never move again; the Life returned the newest value on every one of the 1,711 revised-key probes.

Table 9. The Life beside each shape: exactness, abstention and revision, with the map's paraphrase recall, 20 seeds, 60 turns, $k = 6$.

Measure	Flat + Life	Tree + Life	Cascade + Life
Exact by key (fraction of keyed probes)	1.000 of 5,665	1.000 of 5,665	1.000 of 5,665
False ABSTAIN on a known key (fraction)	0.000	0.000	0.000
ABSTAIN on unknown keys (fraction of probes)	1.000 of 720	1.000 of 720	1.000 of 720
Newest value on revised keys (fraction of probes)	1.000 of 1,711	1.000 of 1,711	1.000 of 1,711
Paraphrase recall with and without the Life (fraction)	0.479 and 0.479	0.578 and 0.578	0.550 and 0.550
Sent per turn with and without the Life (tokens)	461 and 461	461 and 460	461 and 461

The tool-recovering model: what it removes and what it costs (RQ4)

Under a permissive gate that switched after 30 keyed examples, the tool model cut tokens per turn on turns 31 to 60 from 461, with the tools, to 175, and round trips from 2 to 1, with paraphrase recall at 0.478 against 0.479 with the tools (Table 10). It also filed facts under the wrong key: recovered teach keys were correct 0.460 of the time while the model committed to a key 0.935 of the time, and the Life's exact recall fell from 1.000 to 0.734, with false abstentions at 0.111. Tested alone on unseen facts, the key head was right on 11 to 14 of 30 after 30 examples and 26 to 28 of 30 after 120, while its softmax confidence committed on 28 to 30 of 30 at every accuracy (Fig. 5). Under the prequential gate at 0.85, over 120 turns and 20 seeds, the model took over on 0 of 20 seeds; keys predicted from facts ended at a prequential accuracy of 0.663 and keys predicted from questions at 0.948. The prequential figure averages over every example since the first, so it lags the head's current accuracy, which on unseen facts had reached 0.900 after 120 examples.

Table 10. The tool-recovering model: what a permissive gate removes and what it misfiles, 20 seeds, 60 turns, $k = 6$; and the prequential gate over 120 turns.

Measure	Tools every turn	Recovered, permissive gate
Sent to the frontier per turn, turns 31 to 60 (tokens)	461	175
Round trips per turn (count)	2	1
Paraphrase recall at the last turn, mean of seeds (fraction)	0.479	0.478
Recovered teach keys correct (fraction)		0.460
Recovered teaches committed to a key (fraction)		0.935
Life exact by key (fraction)	1.000	0.734
False ABSTAIN on known keys (fraction)	0.000	0.111
Newest value on revised keys (fraction)	1.000	0.638
Prequential gate 0.85, 120 turns: seeds on which it took over (count of 20)		0
Prequential gate: final accuracy, keys from facts and from questions (fraction)		0.663 and 0.948

Tools every turn: the same runs with the tool model observing but never switching. Permissive gate: the switch threshold set to 0, so the model takes over after 30 keyed examples. Sent tokens compare the same turns, 31 to 60: turns 1 to 30 are identical in both runs, so the tools-every-turn mean over turns 31 to 60 follows from its mean over all 60 turns.

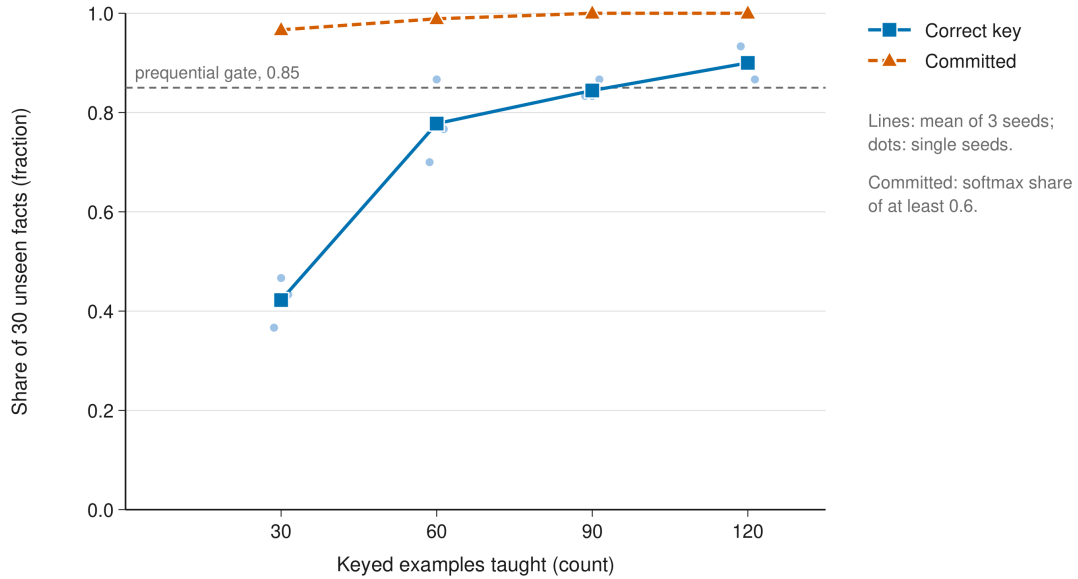


Figure 5. Accuracy and confidence of the key head on unseen facts. The key head was taught n keyed examples and tested on the next 30 facts of the same world, for three seeds. Lines are means and dots single seeds. Committed counts predictions whose softmax share was at least 0.6; the dashed line is the prequential gate at 0.85.

A human-driven frontier, 12 turns

With Claude writing the recall queries and keyed teach pairs and then answering ten probes from the readouts alone, 11 of the 12 facts that the probes needed came back in the readouts and all 11 were answered, with no wrong quotation and no wrong abstention; all 12 keys were exact on recall, and 418 tokens were sent per turn. The one miss was a topic with two facts of which the readout held one. Cues written by a model, richer than the scripted frontier's bare questions, gave a higher readout recall than the synthetic runs, which suggests that the frontier's cue quality is a lever that the synthetic world understates. This is one seed and a sanity check only.

A real transcript, costed

The real session ran to 243,167 estimated tokens over 34 turns of the person, a mean of 7,152 tokens appended per turn and one turn of 90,579. Applying the per-turn cost model to its actual sizes, with the rules at 164 tokens, a readout of four facts and one tool call, the transcript path would have sent 5,547,550 input tokens without a cache, or 831,269 token-equivalents with a warm cache, and the CMLM path 17,816, between 500 and 538 per turn (Table 11, Fig. 6). The CMLM path is cheaper than the cached transcript on every turn except turn 1, because every later turn begins with the first turn's 90,579 tokens of history to read and the previous turn's output to write. By Corollary 1 the session would cross a million-token window at about turn 140 at its mean pace, and at turn 12 at the pace of its largest turn. This applies the cost model to real sizes; it is not a live run, and it says nothing about what the map would have recalled from this transcript.

Table 11. A real 34-turn session, costed with the per-turn model of the paper.

Measure	Value
Turns of the person (count)	34
Transcript at the end (tokens)	243,167
Mean appended per turn; largest single turn (tokens)	7,152; 90,579
Input over the session, transcript without a cache (tokens)	5,547,550
Input over the session, transcript with a warm cache (token-equivalents)	831,269
Input over the session, CMLM path (tokens)	17,816
CMLM path per turn, lowest to highest (tokens)	500 to 538
Turns on which the warm transcript was cheaper than the CMLM path	1
Turn at which a one-million-token window would be crossed, at the mean pace and at the largest turn's pace	140 and 12

Sizes are those of the conversation in which this work was developed; no message text is used. Token counts are characters divided by four. Warm cache: history read at 0.1 and the last turn written at 1.25 times the input price.

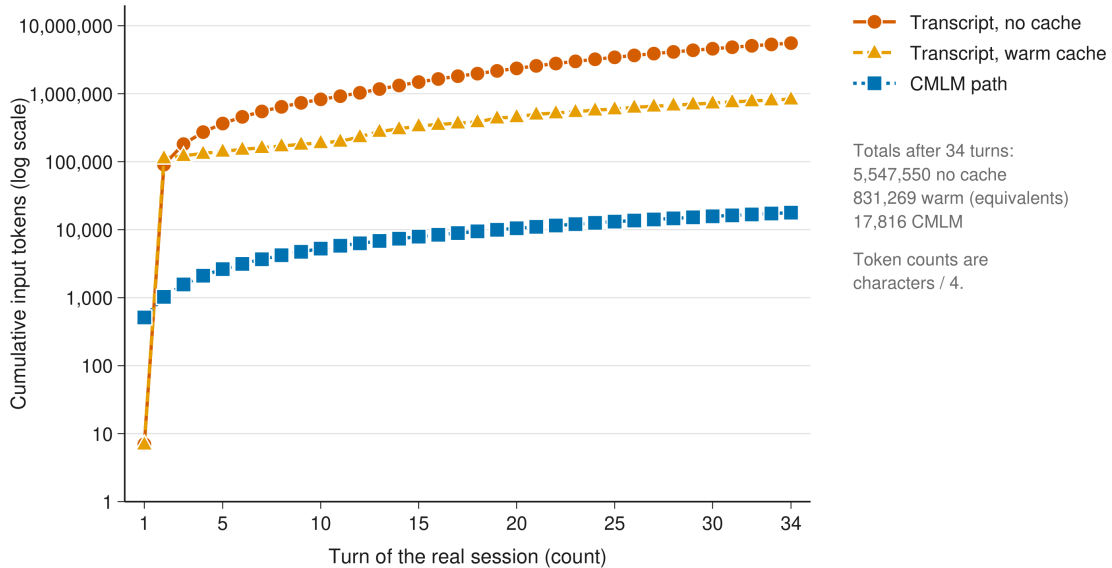


Figure 6. Cumulative input tokens of a real 34-turn session under three paths. The paper's cost model applied to the per-turn sizes of the conversation in which this work was developed. Log scale; token counts are characters divided by four. No message text is used.

Discussion

Where the saving is. Prompt caching decides it. Inside one warm session the transcript path reads its history at a tenth of the input price, so at 1,584 tokens of transcript the CMLM path, with two uncached round trips, costs more input: 429 tokens against about 231 token-equivalents.

It wins past a few thousand tokens of transcript, across sessions, after any gap longer than the cache lifetime, past the window, where the transcript path does not exist, and, by Proposition 6, from the second turn of any session whose turns return documents. Output tokens are unchanged. The design is therefore for deep search, not for short chat under the window.

What theory does not give. Three things are settled only by measurement. Forgetting: a single CMLM loses old facts as it fills, and bounding the store per model is what holds recall. Retrieval precision: a hashed feature space collides, and a fact can come back for the wrong question, which is why verbatim facts live in a store and the weights hold only the map. The teacher's choice: the memory can return only what was written into it, and what is worth writing has no theoretical answer.

A model cannot see its own forgetting. A model's recall of the cues it was taught stayed high while its recall of paraphrases of those cues fell, and the same blindness made the key head commit on 28 to 30 of 30 predictions at every accuracy. The self-measured triggers that were tried, closing a child on its own recall and handing a fact down when its own cue stopped returning it, did no better than a fixed cap, and the key head's own confidence was no guide to its accuracy. A held-out cue per fact, written by the frontier, would give the model a way to re-ask in other words.

Tree versus cascade. The tree is higher because a closed child's weights are clean; the cascade needs no router, orders itself by age and speaks to its own context in vectors. The cascade's training cost per turn grows with depth, since every hand-down trains the level below: at 300 turns a session took 95.2 s against 35.7 s for the tree and 26.4 s for the flat model. Which is right depends on what a heavier substrate does with a frozen weight set.

The latent link. Between two CMLMs nothing is tokenized. Into a frontier served by an API the last hop must be words. With an open-weights frontier the link could be real, and its pieces exist: a trained projector into the model's embedding space for the readout (Alayrac et al., 2022; Liu et al., 2023) and gist tokens for the fixed instructions (Mu, Li & Goodman, 2023). What remains to be measured is whether a projected vector carries a fact as faithfully as its words.

The honest gate. The tool-recovering model shows both edges of the design. The overhead it targets is real, 62% of tokens per turn and a round trip, and so is the cost of taking it over early, measured as misfiled keys. Under a gate that predicts before it learns, the model in this world was not ready within 120 turns. The result is not that the overhead cannot be removed but that it cannot be removed safely by a hashed two-tower model with about a hundred keyed examples, and the record states the accuracy at which it would be safe to try.

Limitations

Internal validity. The frontier is scripted: it asks with the question's words, quotes the readout back and teaches the fact under its true key, so the frontier's own judgement, cue quality, re-asking and abstention are not measured. The 12-turn human-driven run had its cues written by the same session that had seen the facts; only its answering was blind.

External validity. The world is synthetic: one fact per answer, one value per fact, disjoint vocabularies and filler without facts. Real answers state several facts in varied words, and a pattern teacher that keeps sentences with numbers would miss most of them. One substrate, a hashed two-tower linear model, was tested, and a heavier model may move every number in either direction. The live path with a real frontier through an API has not been run.

Construct validity. Recall@k is read against a ceiling that depends on k and on facts per topic, so both raw and ceiling-normalised values are given. Forgetting is operationalised as recall of facts taught more than 15 turns earlier. Token counts are character-based estimates; the API's usage fields are the truth, and the cost analysis inherits the estimate's error, including whether the rules-and-tools prefix reaches a provider's minimum cacheable length.

Conclusion validity. Five seeds at 300 turns, twenty at 60 to 120 turns, forty at 30 and 60 turns and three for the key head's curve. Means are given with standard deviations, 95% confidence intervals and per-seed paired comparisons for the central claims.

Conclusions

A frontier model can keep no transcript and still answer, if its context is a small model that it trains and queries rather than text that it re-sends. The limit on what such a context holds is training, not tokens, and the answer to that limit is expansion rather than a larger window. In a synthetic world that model held what the frontier taught it, forgot as it filled, and stopped forgetting once it expanded into a tree or a cascade of bounded models that talk to each other in vectors. An exact integer memory beside every level made keyed facts verbatim, revisable and refusable at any depth without touching the map. At the scale the design is for, three hundred documents and a million tokens of transcript that no request could carry, the frontier read about 560 tokens a turn and the expanding shapes kept most of the recall that their readout size allowed. The overhead of the tools themselves can be learned away, but not safely by this substrate on about a hundred examples. The next step is the one this paper could not take: the same loop with a real frontier behind the tools, its usage fields as the token counts, and its own cues and keys as the teacher.

Acknowledgements

Use of generative AI. The prototype code, the experiments, the figures and drafts of this manuscript were produced with Claude (Anthropic), using the models Claude Fable 5.1 and Claude Opus 5.5 through Claude Code version 2.1.92, working under the author's direction. In the human-driven frontier experiment, Claude Fable 5.1 acted as the frontier model. The tools were used to write and run code, to draft and revise text, and to check references against arXiv and Crossref, because the author developed the work in conversation with the model. The author conceived the method, directed every experiment, reviewed all content, confirmed its originality and accuracy, checked the terms of use of the tools, and takes full responsibility for the integrity of the whole manuscript, including the references.

References

- Alayrac J-B, Donahue J, Luc P, Miech A, Barr I, Hasson Y, Lenc K, Mensch A, Millican K, Reynolds M, Ring R, Rutherford E, Cabi S, Han T, Gong Z, Samangooei S, Monteiro M, Menick J, Borgeaud S, Brock A, Nematzadeh A, Sharifzadeh S, Binkowski M, Barreira R, Vinyals O, Zisserman A, Simonyan K. 2022. Flamingo: a visual language model for few-shot learning. In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Available at <https://arxiv.org/abs/2204.14198>.
- Behrouz A, Zhong P, Mirrokni V. 2024. Titans: learning to memorize at test time. *arXiv preprint*. Available at <https://arxiv.org/abs/2501.00663>.
- Chevalier A, Wettig A, Ajith A, Chen D. 2023. Adapting language models to compress contexts. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3829–3846. DOI: 10.18653/v1/2023.emnlp-main.232.
- Dawid AP. 1984. Present position and potential developments: some personal views. *Statistical theory: the prequential approach*. *Journal of the Royal Statistical Society, Series A (General)* 147(2):278–292. DOI: 10.2307/2981683.
- Deng C, Zhang Z, Mao K, Li S, Huang X, Yu D, Dou Z. 2024. A silver bullet or a compromise for full attention? A comprehensive study of gist token-based context compression. *arXiv preprint*. Available at <https://arxiv.org/abs/2412.17483>.
- Hardt M, Sun Y. 2023. Test-time training on nearest neighbors for large language models. *arXiv preprint*. Available at <https://arxiv.org/abs/2305.18466>.
- He Z, Karlinsky L, Kim D, McAuley J, Krotov D, Feris R. 2024. CAMELoT: towards large language models with training-free consolidated associative memory. *arXiv preprint*. Available at <https://arxiv.org/abs/2402.13449>.
- Huang PS, He X, Gao J, Deng L, Acero A, Heck L. 2013. Learning deep structured semantic models for web search using clickthrough data. In: *Proceedings of the 22nd ACM International Conference on Information and Knowledge Management (CIKM 2013)*. 2333–2338. DOI: 10.1145/2505515.2505665.

- Kancheti D. 2026. living-fused: a small deterministic memory for local language models. Available at <https://github.com/devkancheti4-design/living-fused> (accessed 28 September 2026).
- Khandelwal U, Levy O, Jurafsky D, Zettlemoyer L, Lewis M. 2020. Generalization through memorization: nearest neighbor language models. In: 8th International Conference on Learning Representations (ICLR 2020). Available at <https://arxiv.org/abs/1911.00172>.
- Lester B, Al-Rfou R, Constant N. 2021. The power of scale for parameter-efficient prompt tuning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. 3045–3059. DOI: 10.18653/v1/2021.emnlp-main.243.
- Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W-t, Rocktäschel T, Riedel S, Kiela D. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS 2020). Available at <https://arxiv.org/abs/2005.11401>.
- Li A, McLeish S, Chen H, Kalra N, Chen Z, Gazizov A, Morisette VASK, Kailkhura B, Menon H, Liu Z, Bartoldson BR, Goldstein T, Lotfi S, Goldblum M, Izmailov P. 2026. End-to-end context compression at scale. arXiv preprint. Available at <https://arxiv.org/abs/2606.09659>.
- Li XL, Liang P. 2021. Prefix-tuning: optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). 4582–4597. DOI: 10.18653/v1/2021.acl-long.353.
- Liu F, Qiu H. 2025. Context cascade compression: exploring the upper limits of text compression. arXiv preprint. Available at <https://arxiv.org/abs/2511.15244>.
- Liu H, Li C, Wu Q, Lee YJ. 2023. Visual instruction tuning. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). Available at <https://arxiv.org/abs/2304.08485>.
- McCloskey M, Cohen NJ. 1989. Catastrophic interference in connectionist networks: the sequential learning problem. *Psychology of Learning and Motivation* 24:109–165. DOI: 10.1016/S0079-7421(08)60536-8.

- Mu J, Li XL, Goodman N. 2023. Learning to compress prompts with gist tokens. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). Available at <https://arxiv.org/abs/2304.08467>.
- Nijkamp E, Koul A, Pakhomov E, Pang B. 2026. An architecture for long-horizon agents: levels, ticks and cascaded intelligence. arXiv preprint. Available at <https://arxiv.org/abs/2609.19519>.
- Oord A van den, Li Y, Vinyals O. 2018. Representation learning with contrastive predictive coding. arXiv preprint. Available at <https://arxiv.org/abs/1807.03748>.
- Packer C, Wooders S, Lin K, Fang V, Patil SG, Stoica I, Gonzalez JE. 2023. MemGPT: towards LLMs as operating systems. arXiv preprint. Available at <https://arxiv.org/abs/2310.08560>.
- Rebuffi SA, Kolesnikov A, Sperl G, Lampert CH. 2017. iCaRL: incremental classifier and representation learning. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 5533–5542. DOI: 10.1109/CVPR.2017.587.
- Schmidhuber J. 1992. Learning to control fast-weight memories: an alternative to dynamic recurrent networks. *Neural Computation* 4(1):131–139. DOI: 10.1162/neco.1992.4.1.131.
- Sun H, Zeng S. 2025. Hierarchical memory for high-efficiency long-term reasoning in LLM agents. arXiv preprint. Available at <https://arxiv.org/abs/2507.22925>.
- V P S. 2026. SR-TTT does not learn retrieval: a correction and mechanistic post-mortem of surprisal-aware residual test-time training. arXiv preprint. Available at <https://arxiv.org/abs/2603.06642>.
- Wang Z, Dang X, Zhu R-J, Wen Z, Fu H, Chai W, Lee JD. 2026. Learning what to remember: test-time training via context distillation. arXiv preprint. Available at <https://arxiv.org/abs/2608.01672>.
- Weinberger K, Dasgupta A, Langford J, Smola A, Attenberg J. 2009. Feature hashing for large scale multitask learning. In: Proceedings of the 26th Annual International Conference on Machine Learning (ICML 2009). 1113–1120. DOI: 10.1145/1553374.1553516.

- Weston J, Chopra S, Bordes A. 2015. Memory networks. In: 3rd International Conference on Learning Representations (ICLR 2015). Available at <https://arxiv.org/abs/1410.3916>.
- Yu Y, Yao L, Xie Y, Tan Q, Feng J, Li Y, Wu L. 2026. Agentic memory: learning unified long-term and short-term memory management for large language model agents. arXiv preprint. Available at <https://arxiv.org/abs/2601.01885>.
- Zhang J, Zhang C, Chen S, Huang Z, Zheng P, Wang Z, Guo P, Mo F, Bae S-H, Zou J, Wei J, Yang Y. 2026a. Lightweight LLM agent memory with small language models. arXiv preprint. Available at <https://arxiv.org/abs/2604.07798>.
- Zhang Z, Xu M, Sun F, Zhang Z. 2026b. OoO-Spec: out-of-order semantic speculation for fast tool calling. arXiv preprint. Available at <https://arxiv.org/abs/2608.00814>.
- Zhao T, Jones L. 2026. Fast-weight product key memory. arXiv preprint. Available at <https://arxiv.org/abs/2601.00671>.